Game Playing Search: Conceptual Insights into Minimax and Alpha-Beta Pruning Optimization

***Abstract*— Adversarial search algorithms are fundamental to decision-making in competitive, zero-sum environments. This report presents the Minimax algorithm and its optimized variant, Alpha-Beta Pruning. It explains their recursive evaluation mechanisms, decision boundaries, and traversal strategies using a complete binary game tree. A practical example demonstrates how Alpha-Beta Pruning eliminates unnecessary branch evaluations while preserving the optimal Minimax result.**

***Keywords*—** *Adversarial Search, Minimax Algorithm, Alpha-Beta Pruning, Artificial Intelligence, Game Tree.*

I. INTRODUCTION

In competitive computing environments, adversarial search models the decision tree of games where multiple players possess directly conflicting goals. Such formulations typically isolate a two-player scenario consisting of an agent labeled MAX (the maximizer) and an opponent labeled MIN (the minimizer). The complete operational layout is visualized as a structured game tree where alternative layers correspond to turns taken by individual players, moving downwards toward fixed terminal depths populated with deterministic utility values.

II. THE MINIMAX ALGORITHM

The Minimax paradigm provides a foundational recursive approach to compute absolute optimal choices by anticipating flawless play from both opponents. The maximizer (MAX) queries child states to secure the largest available metric, while the minimizer (MIN) intentionally limits damage by selecting the absolute lowest available score.

Mathematically, the calculation pattern propagates values upwards from leaf nodes according to alternating tier criteria:

- At a MAX level: Value = max(Left Child, Right Child)
- At a MIN level: Value = min(Left Child, Right Child)

While guaranteed to establish an absolute optimal sequence path, classic Minimax suffers from extreme resource constraints. Because it demands exhaustive inspection of every single node down to the maximum depth level, its absolute time complexity expands exponentially as O(b^d), where b denotes the branching factor and d represents total tree depth.

III. ALPHA-BETA PRUNING OPTIMIZATION

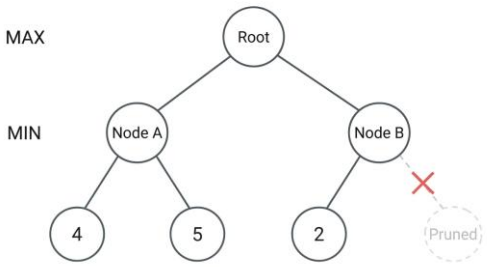
Alpha-Beta Pruning enhances Minimax by skipping search paths that cannot impact the final decision. This optimization guarantees identical value outcomes while saving substantial processing power.

The mechanism tracks two boundary variables throughout the recursive traversal:

1) Alpha (α): The highest value MAX is guaranteed so far across the current branch selection.
2) Beta (β): The lowest value MIN is guaranteed so far across the current branch selection.

As nodes are checked from left to right, parameters are updated continuously. If a step creates a state where $\alpha \geq \beta$, the current player recognizes that the opposing player could force a more disadvantageous outcome earlier in the tree. Thus, further evaluation of that specific branch stops immediately, initiating a prune cutoff.

IV. STRUCTURAL LAYOUT TRACE



To clarify execution flow, consider a complete binary tree with a depth of d = 2. The structural diagram below traces node propagation:

Fig. 1. Execution trace of Alpha-Beta pruning on a depth-2 complete binary game tree. The rightmost branch is cut off (β-cutoff) because Node B (MIN) evaluates a leaf value of 2, which is already less than the guaranteed value of 4 at Node A (MAX), making further evaluation of Node B's children redundant.

Walkthrough Example: Assume the sequential leftmost leaf node scores are [4, 5, 2, 7]. Evaluation proceeds as follows:
1) Node A (MIN) reads Leaf 1 (4) and Leaf 2 (5), storing a

min value of 4. The root updates its alpha threshold to $\alpha = 4$.

2) Evaluation transitions to the right side. Node B (MIN) evaluates Leaf 3 (2). This sets Node B's current bound to $\beta = 2$.

3) At this precise step, the root's alpha (4) is greater than Node B's beta (2), triggering the relation $\alpha \geq \beta$ ($4 \geq 2$).

4) Since MIN will never choose a path scoring higher than 2 here, and MAX already has a guaranteed 4 elsewhere, Leaf 4 is completely pruned from the computation loop.

V. PERFORMANCE ANALYTICS

A direct comparison highlights the architectural advantages provided by pruning variations during adversarial tree processing.

Evaluation Metric	Classic Minimax	Alpha-Beta Pruning
Final Computed Root Value	Exactly Identical	Exactly Identical
Leaf Node Processing Load	Exhaustive (100% Examined)	Highly Reduced (Up to 50% Saved)
Worst-Case Time Complexity	$O(b^d)$	$O(b^d)$

Best-Case Complexity of Alpha-Beta Pruning:

With optimal move ordering, Alpha-Beta Pruning can reduce the effective time complexity to approximately $O\left(b^{\left\{\frac{d}{2}\right\}}\right)$, where (b) is the branching factor and (d) is the search depth. The actual efficiency depends on the ordering of child nodes and the opportunities for pruning.

VI. CONCLUSION

This practical demonstrates the implementation of Minimax and Alpha-Beta Pruning for adversarial search in two-player games. Both algorithms determine the same optimal decision, while Alpha-Beta Pruning improves search efficiency by eliminating unnecessary branch evaluations. The implementation highlights the importance of recursive game tree evaluation and pruning techniques in developing efficient intelligent decision-making systems.